MUNICIPAL WASTE TRANSPORT PLANNING

NAGARFLOW AI

Project Master Guide

Architecture, local development, authenticated APIs, and delivery status for Scenarios, Depots, Vehicles, Storage Points, Processing Facilities, and the Scenario Validation Engine.

Prepared: October 31, 2026

Source of truth for setup and implemented behavior: repository README and code. Figures quoted here were taken from a passing test run, not estimated.

1. Product overview

NAGARFLOW AI is a React and FastAPI application for municipal waste transportation planning. Authenticated municipal coordinators manage planning scenarios together with the depot, vehicle, storage point and processing facility records those scenarios draw on.

The application uses real database records throughout. It does not seed demonstration depots, vehicles, facilities or storage points, and the dashboard reports genuine zeros rather than sample statistics when nothing is configured.

Approved MVP rules

- A scenario uses exactly one depot and one processing facility.
- A scenario uses exactly two vehicles, each starting empty and making at most one trip.
- A scenario contains between six and ten storage points, and every configured point is mandatory.
- A storage point's entire waste quantity is collected by exactly one vehicle. Waste is never divided between vehicles.
- Processing facility capacity is a single shared limit across both vehicle deliveries, not a per-vehicle allowance.

2. Technology and repository layout

Frontend: React 18, React Router 7, MapLibre GL and Vite under **frontend/**. Backend: FastAPI, Pydantic, SQLAlchemy 2 and Uvicorn under **backend/**. Database: PostgreSQL for local operation, with SQLite used by the backend test suite.

The API version prefix is **/api/v1**. The original unversioned **/auth/*** paths remain mounted for compatibility but are hidden from the generated documentation.

Key files

- **backend/app/main.py** - application assembly, routers and startup schema upgrades.
- **backend/app/services/validation.py** - the scenario validation engine.
- **backend/app/db/migrations.py** - in-place upgrades for records created under earlier schemas.
- **frontend/src/App.jsx** - route table.
- **README.md** - setup and operational notes.

Schema management

Tables are created by **Base.metadata.create_all** at application startup, followed by targeted upgrade functions for tables whose shape changed between steps. There is no Alembic. **create_all** creates missing tables but never alters an existing one, so any future column change needs either an upgrade function or a migration tool.

3. Local development setup

Requirements: Node.js and npm, Python 3.10 or later, and PostgreSQL listening on port 5433.

Database

Confirm what is already serving port 5433 before starting anything. A native PostgreSQL service on that port is a complete setup on its own. Starting the bundled container as well would point the API at a second, empty database and existing accounts would appear to vanish.

```
netstat -ano | findstr :5433      # check first
docker compose up -d db           # only if nothing is listening
```

Backend

```
Set-Location backend
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt -r requirements-dev.txt
uvicorn app.main:app --reload
```

Frontend

```
Set-Location frontend
npm install
npm run dev
```

The API is served at **http://localhost:8000** with interactive documentation at **/docs**. The frontend defaults to the local API and accepts **VITE_API_BASE_URL** as an override.

> Note: Vite may bind only the IPv6 loopback. If http://127.0.0.1:5173 refuses the connection, use http://localhost:5173. Restart the backend after pulling changes; the reloader does not always pick up new modules, and a server left running from before a change answers new routes with 404.

4. Authentication and data access

The auth API supports registration, sign-in, current-user lookup and password reset. Successful sign-in returns a short-lived bearer token. Every scenario, depot, vehicle, storage point, facility, validation and dashboard endpoint requires authentication, and every list and mutation is scoped to the signed-in coordinator.

Passwords are hashed with Argon2. Password reset tokens are 256 bits of cryptographically secure randomness, stored only as a SHA-256 hash, valid for 15 minutes, usable exactly once, and a completed reset invalidates every other outstanding token for that account. The forgot-password endpoint returns an identical response whether or not the address has an account, so it cannot be used to discover registered addresses.

> In development, when SMTP_HOST is empty no real email is sent: each message is written to backend/dev_emails/ and the path is logged. Set a unique JWT_SECRET_KEY before any non-local deployment.

5. Step 5 - Scenario Management

The Scenarios page provides listing, creation, details, editing and deletion. API: **GET/POST /api/v1/scenarios**, **GET/PUT/DELETE /api/v1/scenarios/{id}** and **GET /api/v1/scenarios/options**.

A scenario has a per-coordinator unique name, a description, an active depot, an active processing facility, exactly two different available vehicles, six to ten different storage points, and Draft or Ready status. Selected resource IDs persist through scenario relationship tables.

Rejected at creation

- Unavailable vehicles, and vehicles assigned to a different depot.
- Unavailable storage points.
- Waste totals exceeding the two selected vehicles' combined single-trip capacity.
- Waste totals exceeding the selected facility's shared capacity.
- An inactive processing facility.

A scenario cannot be deleted while an optimization run is queued, pending, running or in progress.

6. Step 6 - Depot Management

The Depot Management page provides listing, add, details, edit and delete. API: **GET/POST /api/v1/depots** and **GET/PUT/DELETE /api/v1/depots/{id}**.

Depot fields are name, unique code, address, latitude, longitude and active status. Coordinates are range-checked. Vehicles require a configured active depot, and inactive depots are excluded from new scenario choices. A depot linked to a scenario or assigned to a vehicle cannot be deleted. Existing database records are upgraded without manufacturing location data.

7. Step 7 - Vehicle Management

The Vehicles page provides listing, add, details, edit and delete. API: **GET/POST /api/v1/vehicles** and **GET/PUT/DELETE /api/v1/vehicles/{id}**.

Fields are a unique registration number, display name, Truck or Mini Truck type, positive capacity stored as a decimal, starting depot, and Available, Maintenance or Inactive status. Vehicle starting coordinates follow the selected depot.

Only available vehicles assigned to a scenario's selected depot can be chosen. A scenario selects exactly two vehicles and cannot exceed their combined capacity. Vehicles linked to scenarios cannot be deleted or made unavailable until the affected scenarios are updated. The fleet inventory may hold more than two vehicles; the two-vehicle rule is per scenario.

8. Step 8 - Storage Point Management

The Storage Point Management page provides listing, add, details, edit, delete, search and filtering, with a MapLibre and OpenStreetMap view. API: **GET/POST /api/v1/storage-points** and **GET/PUT/DELETE /api/v1/storage-points/{id}**.

Fields are name, unique point code, address, latitude, longitude, waste quantity, availability and description. The table carries database CHECK constraints for a non-negative waste quantity and valid coordinate ranges, so invalid values are refused by the database as well as the API.

Relationship safeguards

- A storage point linked to a scenario cannot be deleted, nor marked unavailable, until that scenario is updated or removed.
- A quantity change that would push a linked scenario past its vehicle capacity is refused.
- Invalid points are reported rather than silently excluded from a scenario.

Each map marker shows the point name, code, waste quantity, availability and coordinates.

9. Step 9 - Processing Facility Management

The Processing Facility page provides listing, add, details, edit, delete, activation and deactivation, a shared-capacity monitor and a map view. API: **GET/POST /api/v1/facilities**, **GET/PUT/DELETE /api/v1/facilities/{id}** and **GET /api/v1/facilities/{id}/capacity**.

Fields are name, unique facility code, address, latitude, longitude, total capacity, status and description. Capacity is stored as NUMERIC(12,2) with a database CHECK requiring it to be greater than zero, alongside coordinate range checks. The column retains the name **capacity_kg** while the API exposes it as **total_capacity_kg**.

Shared capacity

Capacity is one pool across both vehicle deliveries. With a 10,000 kg facility, a 4,000 kg delivery by vehicle 1 and a 5,000 kg delivery by vehicle 2 leave 1,000 kg remaining. The constraint is W1 + W2 <= F, where F is the single facility capacity.

Relationship safeguards

- Capacity cannot be reduced below what a linked scenario already commits.
- A facility linked to a scenario can be neither deactivated nor deleted.
- Inactive facilities are not offered when building a new scenario.

> The capacity endpoint reports allocated kilograms, meaning what linked scenarios commit, because no optimization run has recorded a real delivery yet. It becomes the delivered total once the optimization modules land.

10. Step 10 - Scenario Validation Engine

The validation engine is the gate in front of the optimization algorithms. API: **POST /api/v1/scenarios/{id}/validate** runs the checks and records the outcome; **GET /api/v1/scenarios/{id}/validations** returns the saved history, newest first. Both Greedy and OR-Tools must honour exactly these constraints, so the rules live in one module rather than being restated per algorithm.

Checks, in order

- **depot** - exactly one active depot with valid coordinates.
- **vehicles** - exactly two active vehicles.
- **storage_points** - between six and ten points.
- **facility** - exactly one active processing facility.
- **waste_quantity** - no negative quantity.
- **vehicle_capacity** - every whole point fits a single vehicle.
- **combined_capacity** - total waste fits both vehicles together.
- **facility_capacity** - total waste fits the shared facility capacity.
- **coordinates** - every location has valid coordinates.
- **vehicle_assignment** - a split of whole points between the two vehicles exists.
- **road_connectivity** - every required route is reachable by road.

Why the assignment check is separate

Passing the capacity checks does not mean a feasible assignment exists. Three 3,000 kg points against vehicles of 5,000 kg and 4,000 kg total 9,000 kg against a combined 9,000 kg, and each point fits a vehicle on its own, yet no split of whole points works because any two points make 6,000 kg. The engine enumerates every subset, so a negative answer means no assignment exists rather than that the search gave up.

Statuses

- **valid** - every required check passed; optimization is allowed.
- **invalid** - at least one check failed; all failures are reported, not only the first.
- **incomplete** - nothing failed, but a required check could not run.

> Road connectivity needs the OpenStreetMap road network, which arrives in Step 11. Until then the check reports not_checked and the scenario is called incomplete, which blocks optimization. This is deliberate: a scenario is never marked valid on the strength of a check that was never performed. Setting REQUIRE_ROAD_CONNECTIVITY=false is an explicit development override and is logged on every use.

Every validation run is saved with its scenario, status, checks, errors, warnings, waste total and timestamp, so the decision that allowed or blocked optimization can be reviewed later. Reports are recomputed on each request rather than cached, because the underlying records can change at any time.

11. API quick reference

```
GET    /health
POST   /api/v1/auth/register
POST   /api/v1/auth/login
GET    /api/v1/auth/me
POST   /api/v1/auth/forgot-password
POST   /api/v1/auth/verify-reset-token
POST   /api/v1/auth/reset-password
GET    /api/v1/dashboard/summary
GET    /api/v1/scenarios/options
GET    /api/v1/scenarios
POST   /api/v1/scenarios
GET    /api/v1/scenarios/{id}
PUT    /api/v1/scenarios/{id}
DELETE /api/v1/scenarios/{id}
POST   /api/v1/scenarios/{id}/validate
GET    /api/v1/scenarios/{id}/validations
GET    /api/v1/depots
POST   /api/v1/depots
GET    /api/v1/depots/{id}
PUT    /api/v1/depots/{id}
DELETE /api/v1/depots/{id}
GET    /api/v1/vehicles
POST   /api/v1/vehicles
GET    /api/v1/vehicles/{id}
PUT    /api/v1/vehicles/{id}
DELETE /api/v1/vehicles/{id}
GET    /api/v1/storage-points
POST   /api/v1/storage-points
GET    /api/v1/storage-points/{id}
PUT    /api/v1/storage-points/{id}
DELETE /api/v1/storage-points/{id}
GET    /api/v1/facilities
POST   /api/v1/facilities
GET    /api/v1/facilities/{id}
PUT    /api/v1/facilities/{id}
DELETE /api/v1/facilities/{id}
GET    /api/v1/facilities/{id}/capacity
```

Every endpoint except **/health** and the public auth flows requires bearer authentication.

12. Verification and tests

Backend tests run against an in-memory SQLite database, so they need neither Docker nor the development database and never touch development data. Frontend tests render the real application in jsdom and replace only the API module, so routing, the authentication guard and the pages are exercised for real.

```
cd backend   ; py -m pytest
cd frontend  ; npm test
cd frontend  ; npm run build
```

At the time this guide was generated the backend suite passed **153** tests, the frontend suite passed **72** tests, and the Vite production build succeeded. Coverage spans authentication, password reset, CRUD and validation for every resource, relationship persistence, legacy schema upgrades, shared facility capacity, and the validation engine including the assignment check.

Confidence in the tests

Hard constraints were checked by deliberately breaking each rule and confirming that a specific test failed: treating facility capacity as per-vehicle, allowing a capacity reduction below an existing commitment, deleting a linked facility, offering inactive facilities, assuming an assignment always exists, accepting an oversized storage point, treating unchecked road connectivity as a pass, reporting only the first failure, skipping validation history, and enabling the optimization button regardless of the result. Every one was caught.

13. Data and operational safeguards

Never commit **.env** or real credentials. The repository ignores local environment files, node_modules, frontend build output, development email output and database dumps. **.env.example** holds placeholders only, and **JWT_SECRET_KEY** must be unique and private outside local development.

Do not populate resource tables with invented locations or capacities to make a form look complete. Add genuine municipal records through the management pages. Where a figure is unavailable the interface says so, reporting zero or "Not configured" rather than a plausible-looking number.

Known issues

- The dashboard scopes vehicles through scenarios, so a vehicle registered in the fleet but not yet attached to a scenario is reported as zero available. The storage point query in the same place is scoped to the coordinator instead, so the two behave differently.
- The dashboard facility card sums every facility's total capacity, including inactive ones, and does not subtract what scenarios commit, although it is labelled as remaining capacity.

Both are pending a decision on what the dashboard cards should be scoped to: the whole fleet, or only records attached to a scenario.

14. Delivery roadmap

Completed

- Authentication, password reset and protected routing.
- Dashboard with database-backed summary cards and sections.
- Step 5 Scenario Management.
- Step 6 Depot Management.
- Step 7 Vehicle Management.
- Step 8 Storage Point Management, with map view.
- Step 9 Processing Facility Management, with shared-capacity monitoring.
- Step 10 Scenario Validation Engine, with saved validation history.

Next

- Step 11 OpenStreetMap road network integration, which supplies the road connectivity check and replaces straight-line distance with road distance.
- Greedy baseline routing, then Google OR-Tools optimization, both bound by the constraints the validation engine already enforces.
- Algorithm comparison reporting across the S01 to S10 evaluation scenarios.

Open decisions

DEC-003, waste availability timing. Still unresolved. When a storage point's waste is not yet available, it is undecided whether a vehicle waits at the point, skips it and returns later, or the scenario is infeasible. The three choices produce different route distances and are not interchangeable. No time-dependent waiting logic has been implemented; the **available_from** column exists on storage points and the validation engine deliberately ignores it until the policy is settled.

Dashboard card scope. See the known issues in section 13.

Schema migrations. Whether to adopt Alembic or continue with targeted upgrade functions, as described in section 2.